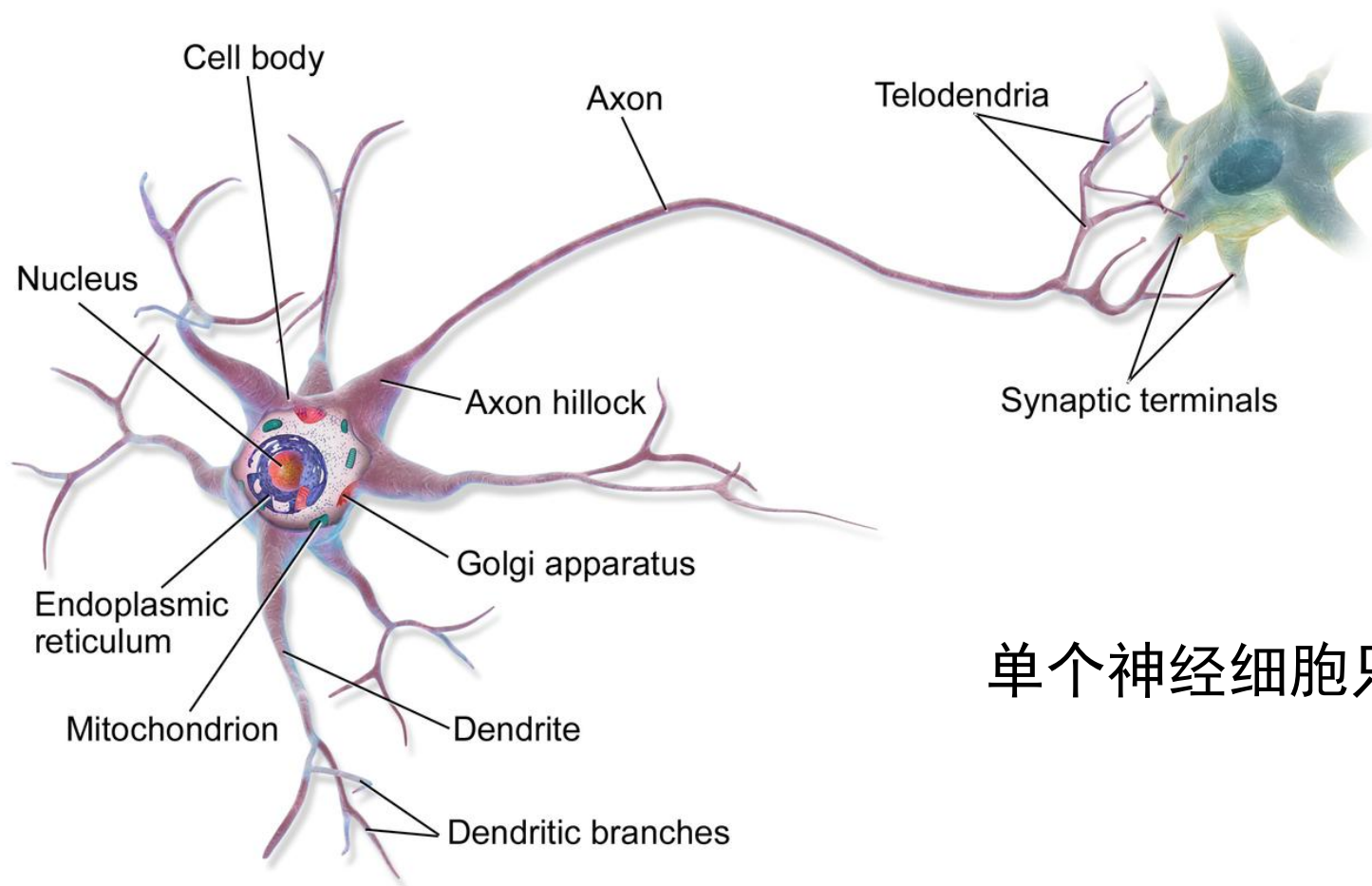


Lecture4

神经网络

生物神经元

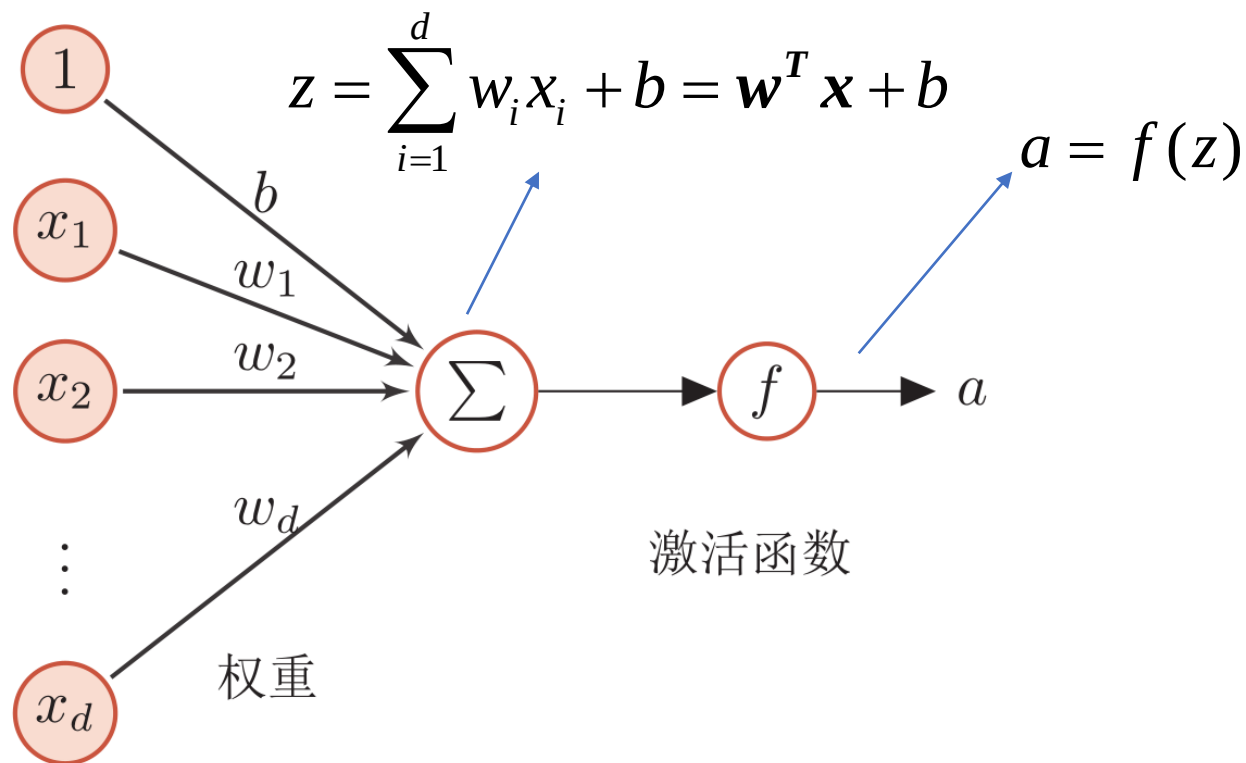


单个神经细胞只有两种状态：兴奋和抑制

人工神经元

■ M-P 神经元模型[McCulloch and Pitts,1943]

神经元接到来自前置 d 个神经元传递过来的输入信号，这些输入信号通过带权重的连接进行传递，神经元接收到的总输入值将与神经元的阈值进行比较，然后通过“激活函数”处理产生神经元的输出。



人工神经网络

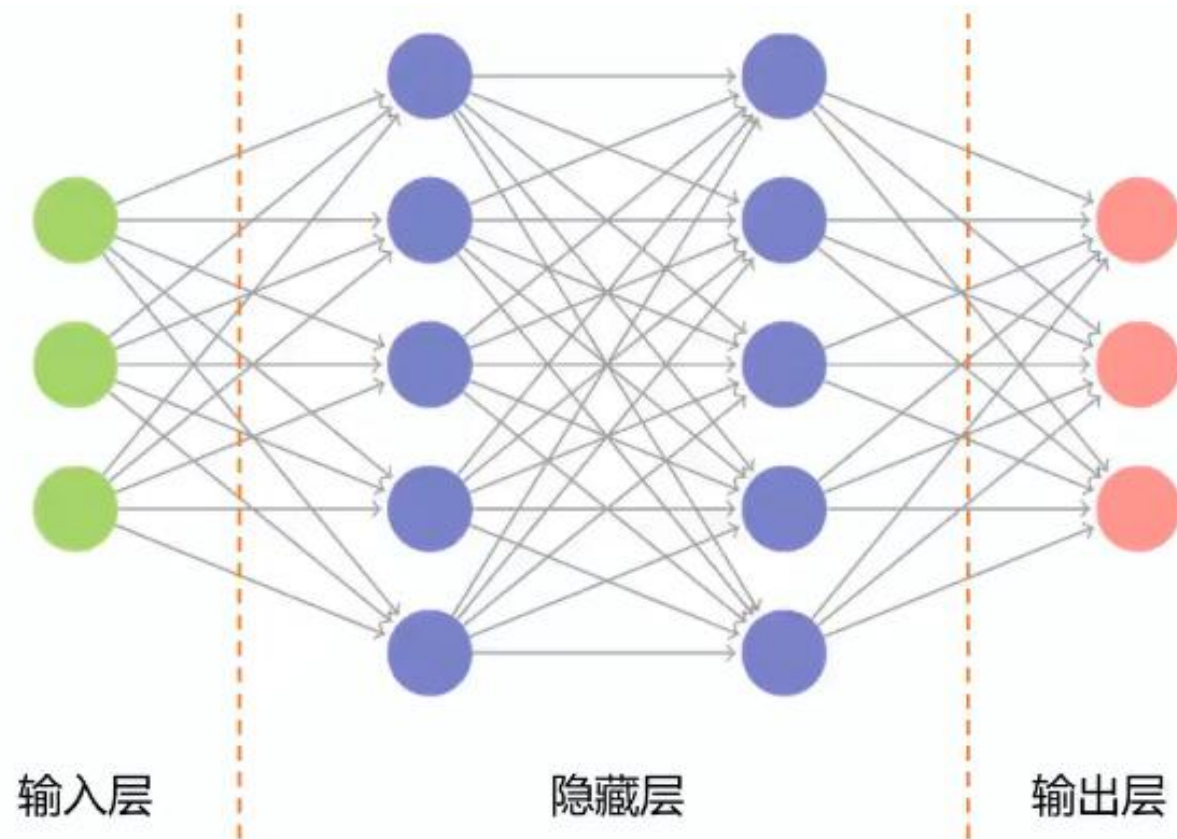
■ 把许多人工神经元按一定的层次结构连接起来，就形成了人工神经网络。

■ 人工神经网络的三大要素：

✓ 节点 —— 采用什么激活函数？

✓ 连边 —— 权重（参数）是多少？

✓ 连接方式 —— 如何设计层次结构？

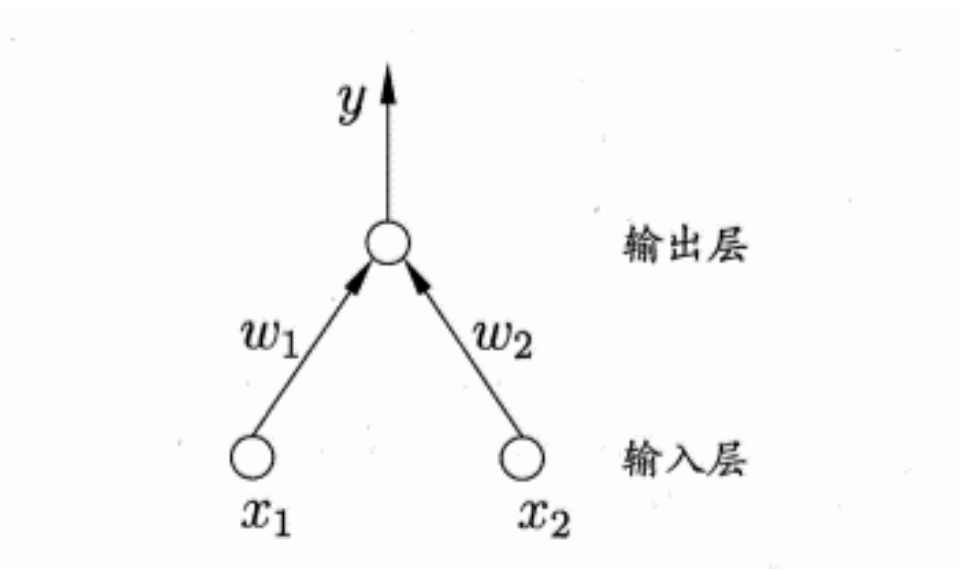


一个解决异或问题的简单网络

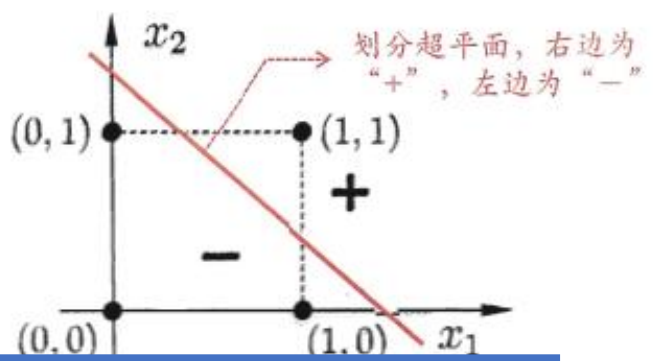
- 感知机回顾
- 双层感知机解决异或问题

感知机求解异、或、非及异或问题

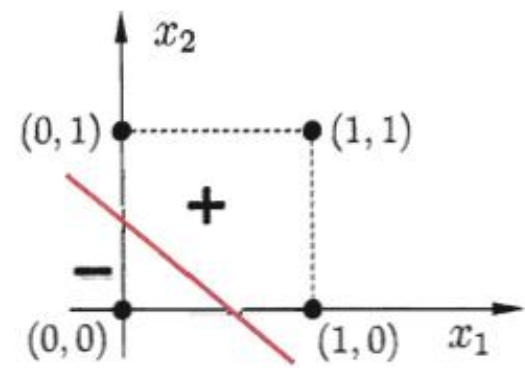
■ 输入为 $[x_1; x_2]$ 的单层单个神经元（输入层不计入层数），采用阶跃激活函数。



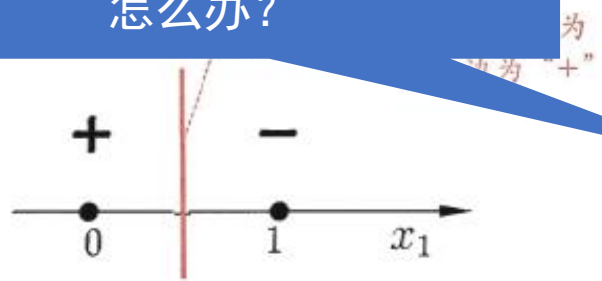
感知机



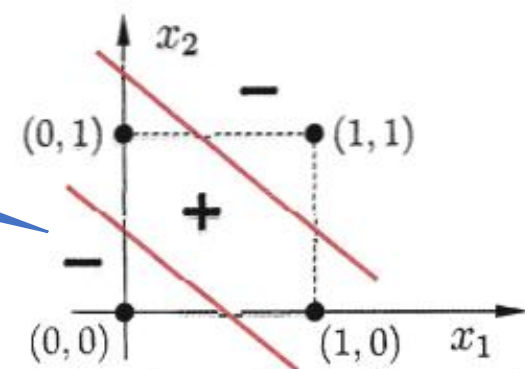
异或问题非线性可分，单层感知机无法解决非线性可分问题，怎么办？



(b) “或”问题 ($x_1 \vee x_2$)



(c) “非”问题 ($\neg x_1$)



(d) “异或”问题 ($x_1 \oplus x_2$)

双层感知机 —— 一个简单的神经网络

■ 输入仍为 $[x_1; x_2]$, 让网络包含两层

✓ 隐藏层包含两个神经元:

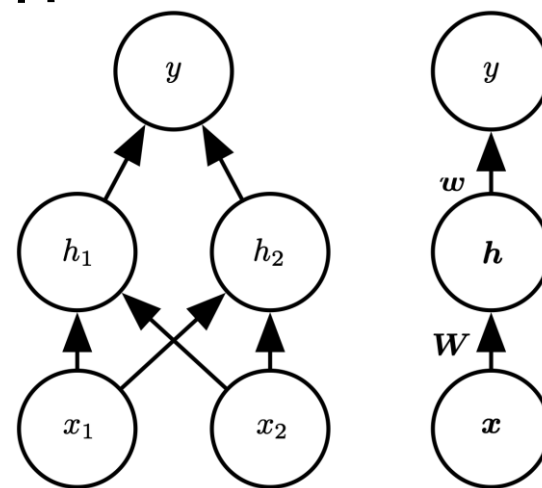
$$\mathbf{h} = f^{(1)}(\mathbf{x}; \mathbf{W}, \mathbf{c})$$

✓ 输出层包含一个神经元:

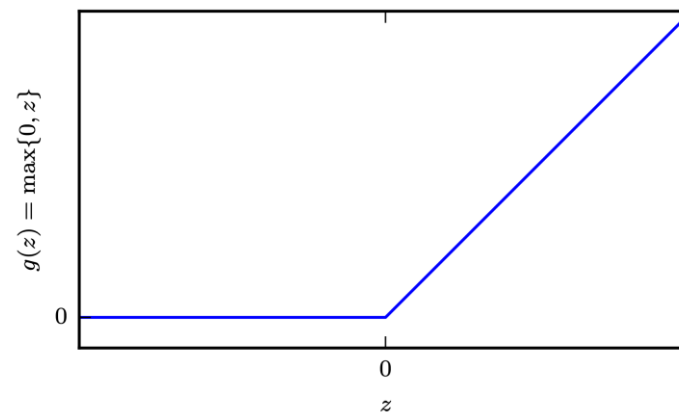
$$y = f^{(2)}(\mathbf{h}; \mathbf{w}, b)$$

✓ 隐藏层采用线性整流激活函数 (ReLU),
则整个模型为:

$$\begin{aligned} f(\mathbf{x}; \mathbf{W}, \mathbf{c}, \mathbf{w}, b) &= f^{(2)}\left(f^{(1)}(\mathbf{x})\right) \\ &= \mathbf{w}^T \max\{0, \mathbf{W}^T \mathbf{x} + \mathbf{c}\} + b \end{aligned}$$



双层感知机



ReLU函数 $g(z) = \max\{0, z\}$

双层感知机 —— 一个简单的神经网络

■ 给出异或问题的一个解：

$$W = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, c = \begin{pmatrix} 0 \\ -1 \end{pmatrix}^T, w = \begin{pmatrix} 1 \\ -2 \end{pmatrix}, b = 0$$

模型处理流程如下：

① 输入4个样本的矩阵表示为：

$$X = \begin{pmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{pmatrix}$$

② 乘以第一层权重矩阵，得到：

$$XW = \begin{pmatrix} 0 & 0 \\ 1 & 1 \\ 1 & 1 \\ 2 & 2 \end{pmatrix}$$

③ 加上偏置向量 c ，得到：

$$XW + c = \begin{pmatrix} 0 & -1 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{pmatrix}$$

④ 使用ReLU激活函数，得到：

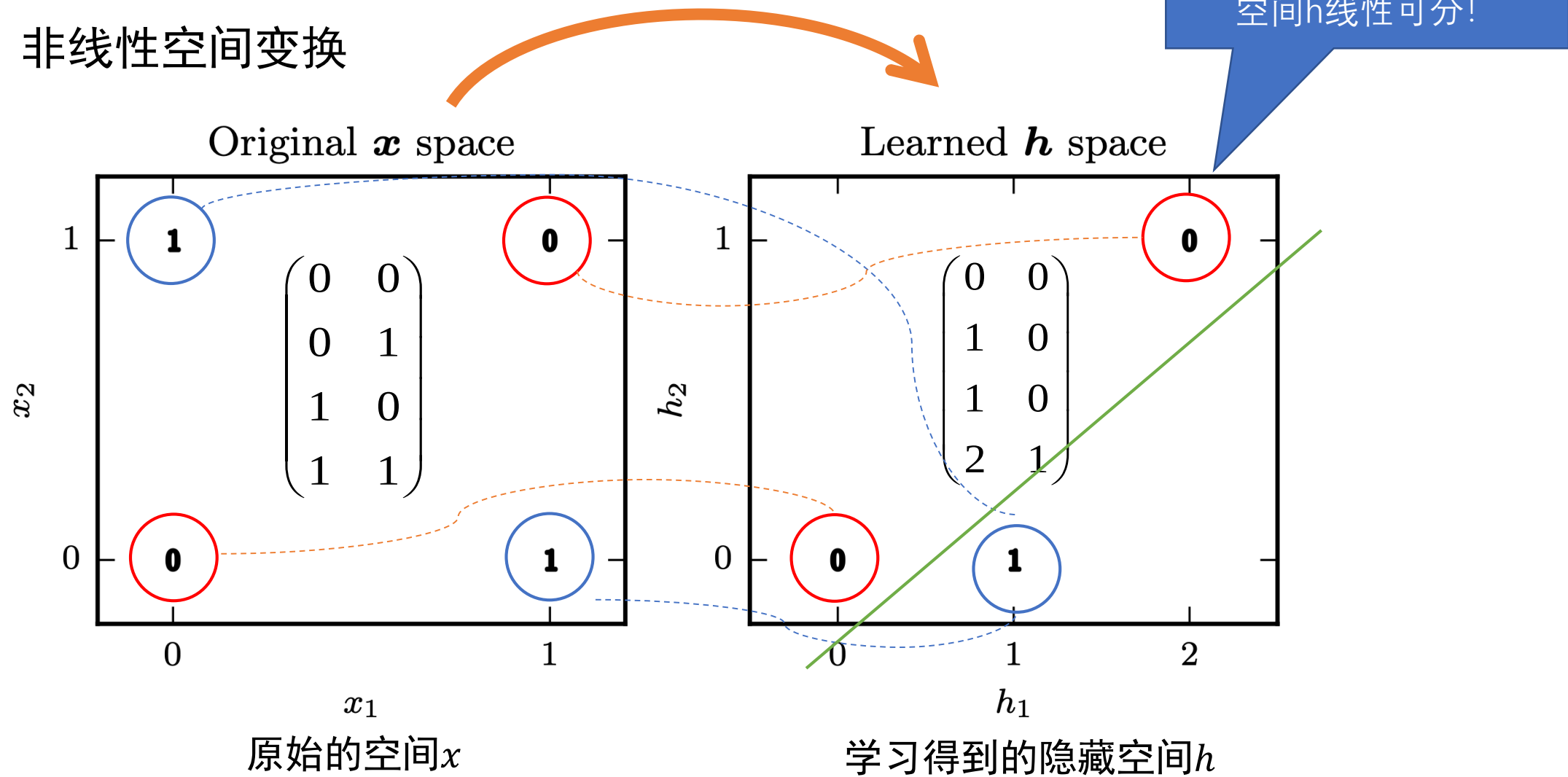
$$\max\{0, XW + c\} = \begin{pmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{pmatrix}$$

⑤ 乘以第二层权重向量 w ，得到：

$$y = \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \end{pmatrix}$$

双层感知机—— 一个简单的神经网络

■ 解释：非线性空间变换



神经网络的结构

- 为什么要加深度
- 常见神经网络结构

为什么要加深度

- 数学理论表明，**单隐层网络**可以近似任何函数，但其**规模可能巨大**
 - ✓ 在最坏情况下，需要指数级别的隐藏单元才能近似某个函数[Barron,1993]
- 随着深度的增加，网络的表示能力呈指数增加
 - ✓ 具有 d 个输入、深度为 l 、每个隐藏层具有 n 个单元的深度学习网络可以描述的线性区域的数量为

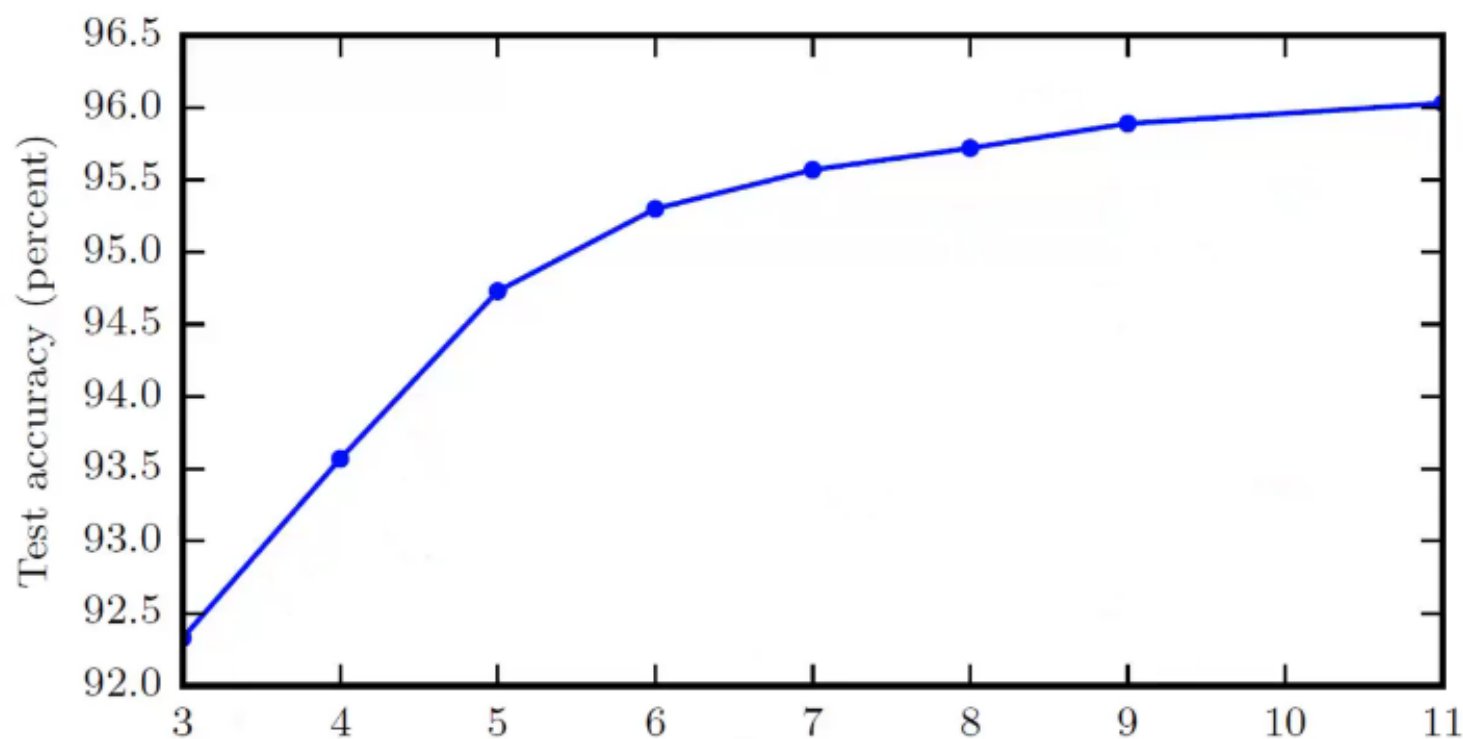
$$O\left(\binom{n}{d}^{d(l-1)} n^d\right)$$

意味着，描述能力为深度的指数级[Montufar et al,2014]。

深度的影响

■ 更深层的网络具有更好的泛化能力

✓ [Goodfellow et al., 2014] 手写数字识别的实验结果

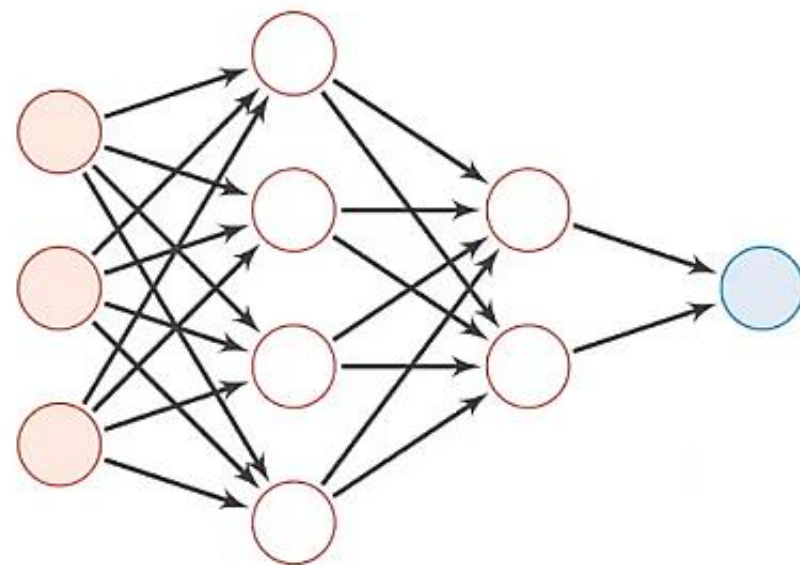


模型的性能随着深度的增加而不断的提升

常见的神经网络结构

■ 前馈网络

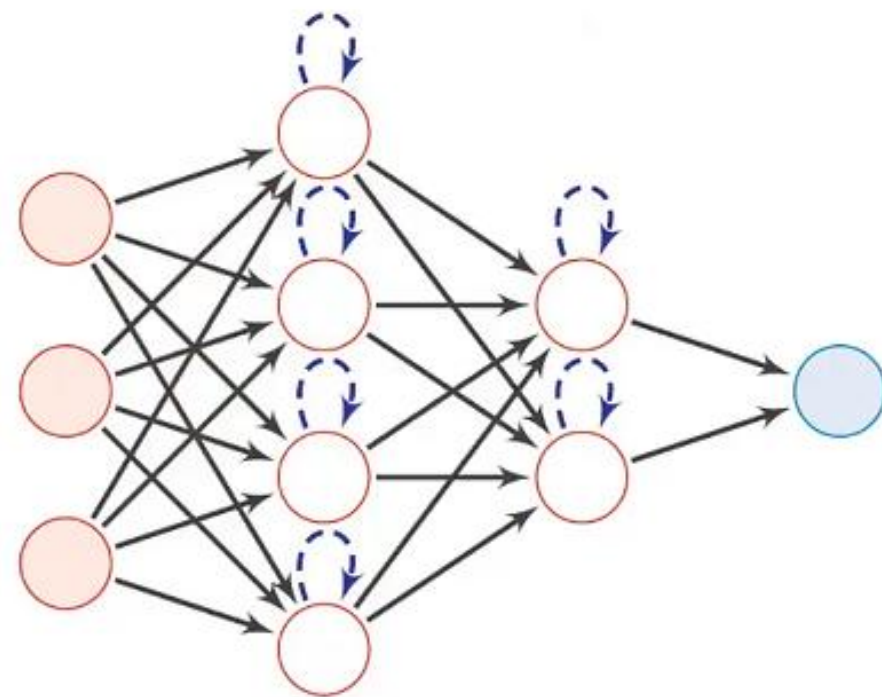
- ✓ 各个神经元按照接收信息的先后分成不同得组，每一组可看做一个神经网络层
- ✓ 每一层中的神经元接收来自前一层神经元的输出，并输出给下一个神经元
- ✓ 整个网络中朝一个方向传播，没有反向的信息传播，可以用一个有向无环图表示



常见的神经网络结构

■ 记忆网络（反馈网络）

- ✓ 神经元不但可以接受其他神经元的信息，也可以接收自己的**历史信息**
- ✓ 神经元具有记忆功能，在**不同时刻具有不同的状态**
- ✓ 信息传播可以是**单向或者双向传递**，可以用一个有向循环图或者无向图来表示
- ✓ 记忆网络包括**循环神经网络（RNN）**、Hopfield网络、Boltzmann机等

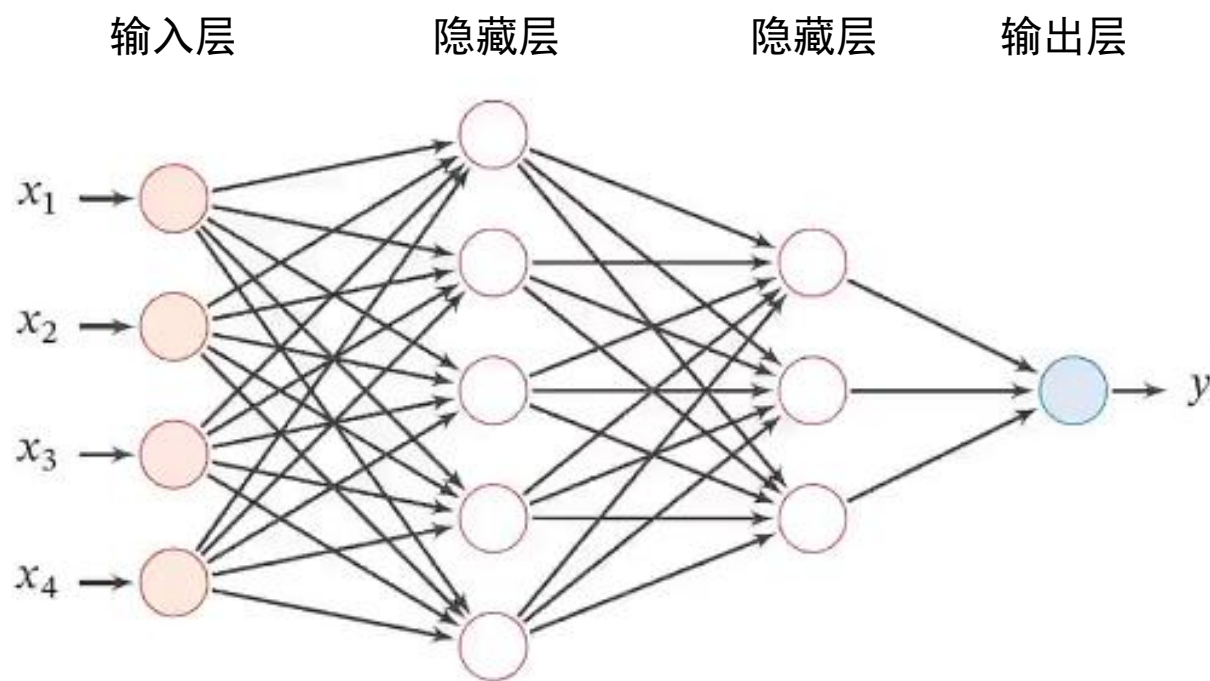


前馈神经网络

- 结构与表示
- 隐藏单元
- 输出单元
- 参数学习

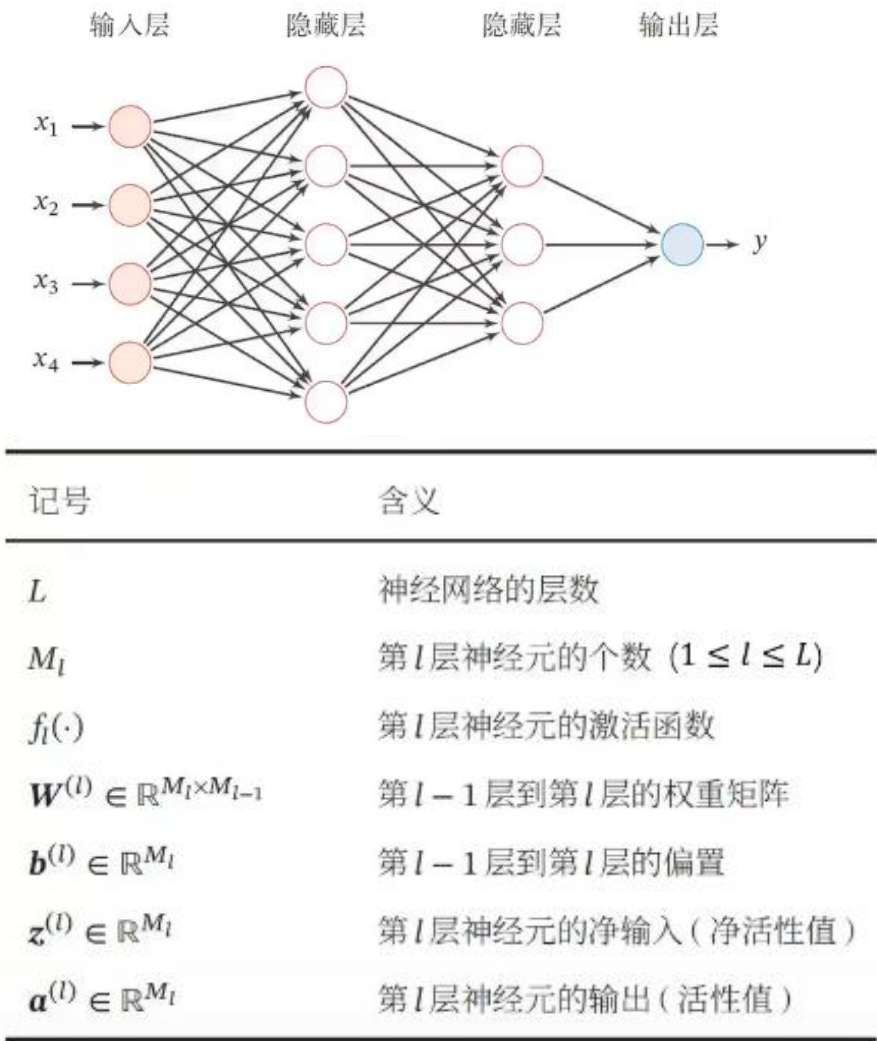
前馈神经网络的结构

- 前馈神经网络 (Feedforward Neural Network, FNN) 是最早发明的简单人工神经网络，也经常被称为**多层感知器 (MLP)** (**激活函数通常并不是感知机所采用的不连续阶跃函数**)
- 第0层为输入层，最后一层为输出层，其他中间层称为隐藏层
- 信号从输入层向输出层单向传播，整个网络无反馈，可用一个**有向无环图**表示



前馈神经网络的形式化表示

前馈神经网络的符号表示



前馈神经网络的信息传递

- ✓ 令 $a^{(0)} = x$ ，信息通过以下公式不断迭代传播：
$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$$
$$a^{(l)} = f_l(z^{(l)})$$
- ✓ 以上公式也可合并写成：
$$a^{(l)} = f_l(W^{(l)} a^{(l-1)} + b^{(l)})$$
- ✓ 如此，通过逐层传递，得到最后的输出： $a^{(L)}$ ，整个网络可以看成是一个复合函数 $\varphi(x; W, b)$

$$a^{(0)} \rightarrow z^{(1)} \rightarrow a^{(1)} \rightarrow z^{(2)} \dots \rightarrow a^{(L-1)} \rightarrow z^{(L)} \rightarrow a^{(L)}$$

隐藏单元——激活函数

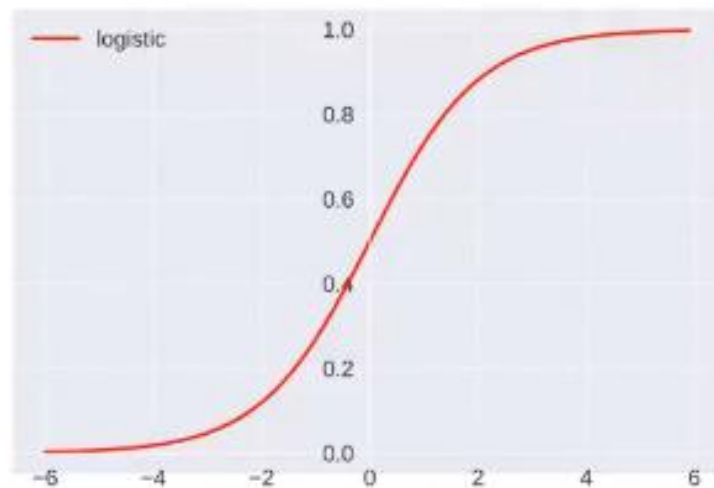
- 隐藏单元的设计是一个非常活跃的研究领域，但是目前还没有很明确的指导原则
- 激活函数的性质要求
 - ✓ 连续并可导 (允许少数点的上不可导) 的非线性函数。可导的激活函数可以直接利用数值优化的方法来学习网络参数。
 - ✓ 激活函数及其导函数要尽可能的简单，有利于提高网络计算效率。
 - ✓ 激活函数的导函数的值域要在一个合适的区间内，不能太大也不能太小，否则会影响训练的效率和稳定性。

常见的激活函数

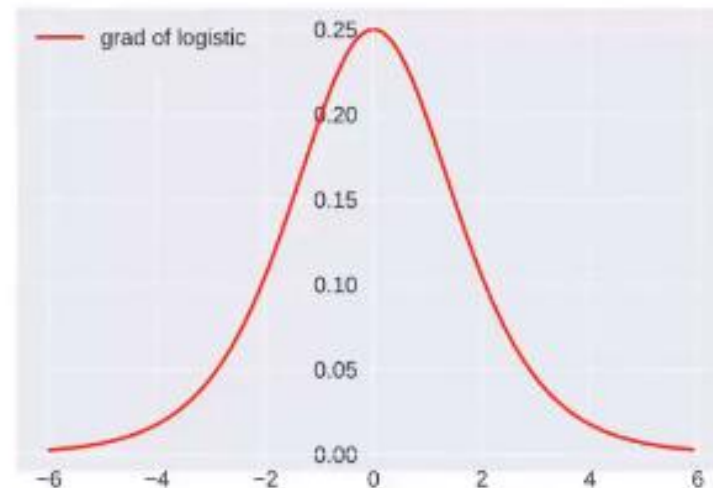
■ Sigmoid函数

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- ✓ 具有“挤压”功能
- ✓ 输出可看作概率分布



Sigmoid函数

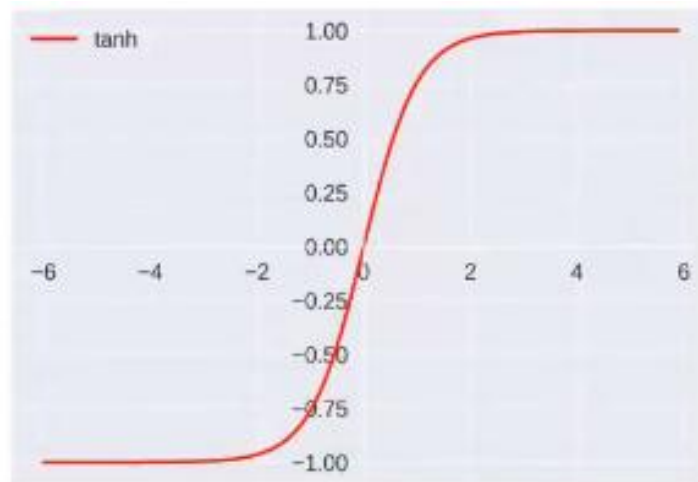


Sigmoid函数的导数

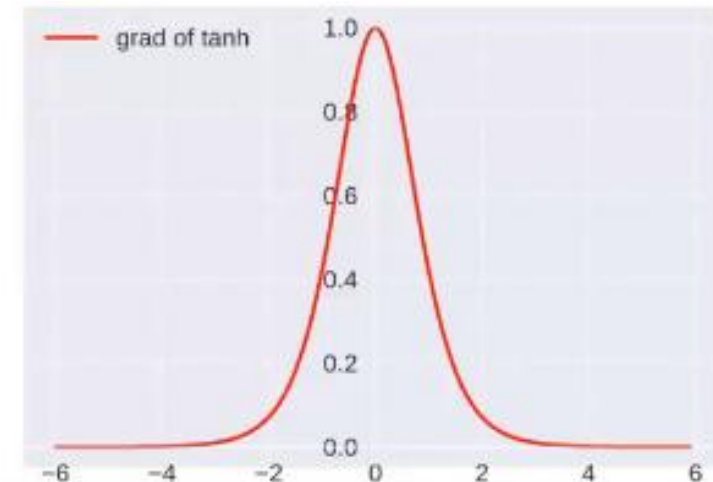
■ Tanh函数

$$\begin{aligned}\tanh(x) &= \frac{e^x - e^{-x}}{e^x + e^{-x}} \\ &= 2\sigma(2x) - 1\end{aligned}$$

- ✓ 零中心化，可提升收敛速度



Tanh函数



Tanh函数的导数

常见的激活函数

■ ReLu函数

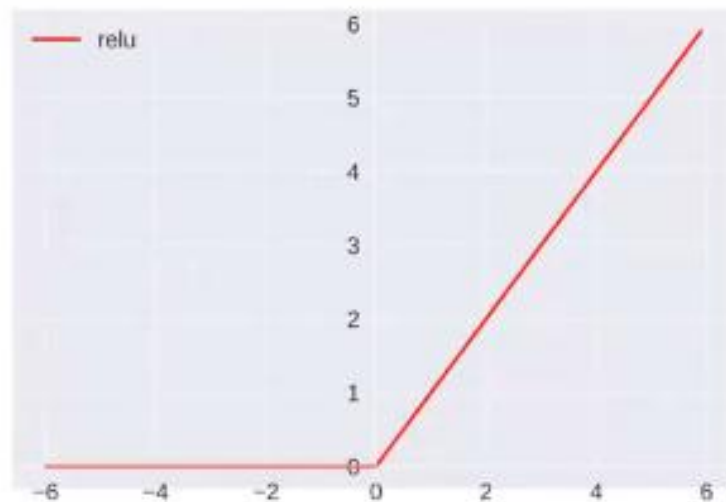
$$\text{ReLu}(x) = \begin{cases} x & x \geq 0 \\ 0 & x < 0 \end{cases} = \max(0, x)$$

- ✓ 目前最常用的激活函数
- ✓ 可缓解梯度消失的问题
- ✓ 缺点：可能导致神经元的死亡

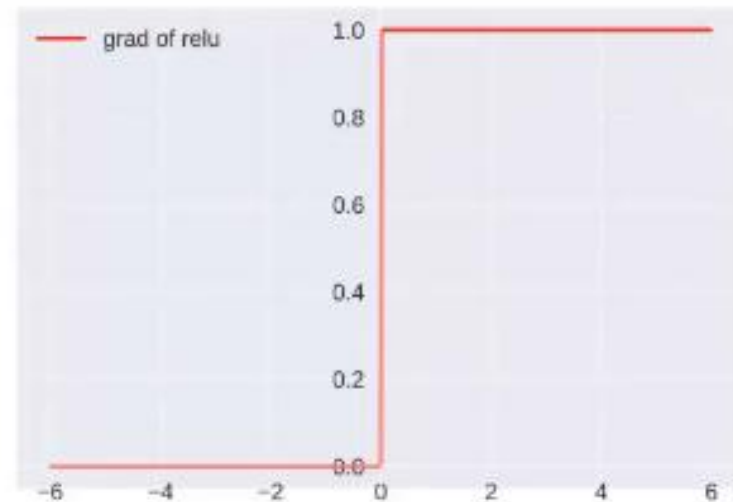
■ LeakReLU

$$\text{LeakReLU}(x) = \begin{cases} x & x \geq 0 \\ \gamma x & x < 0 \end{cases}$$

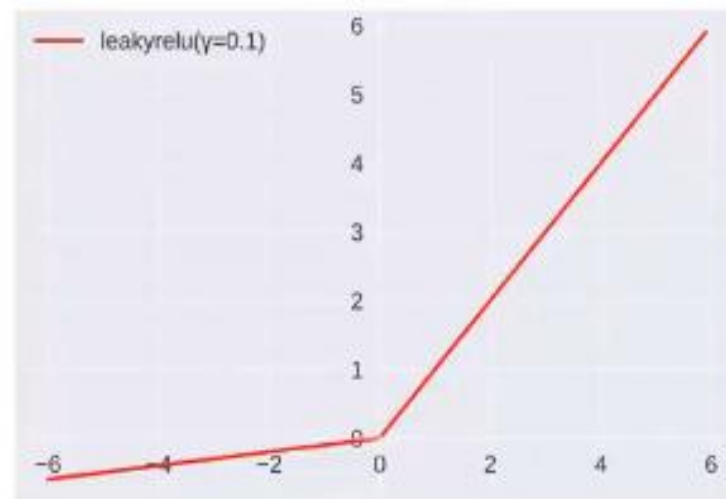
- ✓ 在 $x < 0$ 时也保持一个很小的梯度, 避免了永远不能激活的情况
- ✓ γ 为超参数



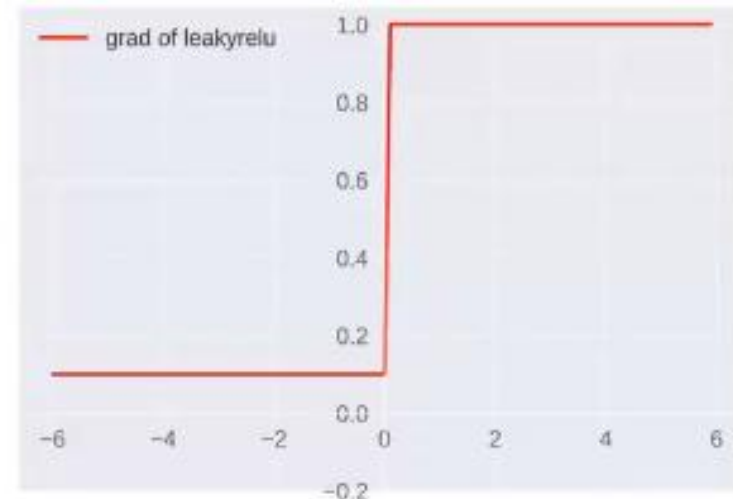
ReLU函数



ReLU函数的导数



LeakReLU函数



LeakReLU函数的导数

输出单元

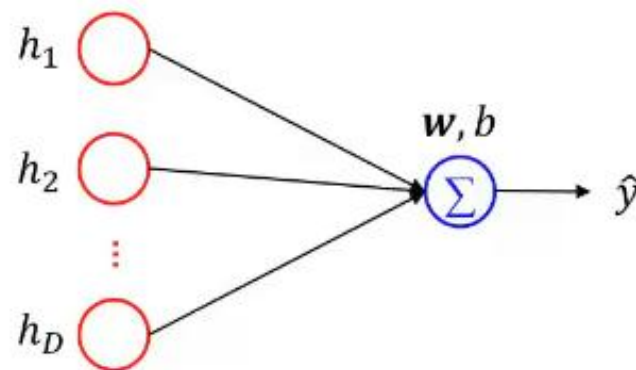
■ 线性输出单元

$$\hat{y} = \mathbf{w}^T \mathbf{h} + b$$

- ✓ 线性输出单元常用于产生条件高斯分布的均值。
- ✓ 适合连续值预测（回归）问题。
- ✓ 基于高斯分布，最大似然（最小化负对数似然）等价于最小化均方误差，因此线性输出单元可采用均方误差损失函数：

$$L(\mathbf{y}, \hat{\mathbf{y}}) = \frac{1}{N} \sum_{n=1}^N \| \hat{y}^{(n)} - y^{(n)} \|^2$$

其中 $y^{(n)}$ 为真实值， $\hat{y}^{(n)}$ 为预测值， N 为样本数。



输出单元

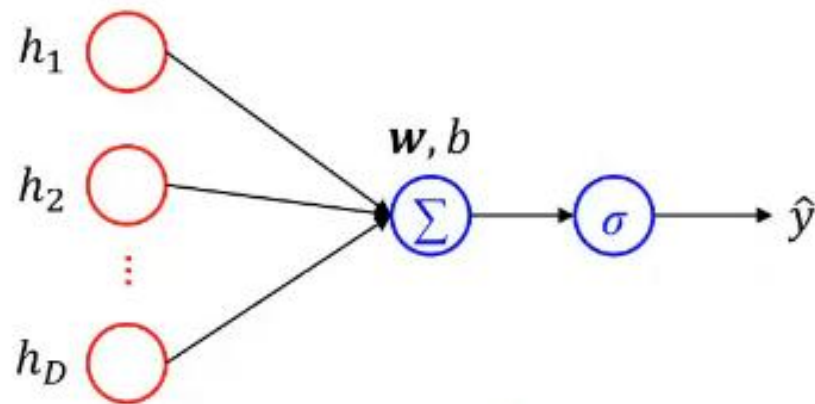
■ Sigmoid单元

$$\hat{y} = \sigma(\mathbf{w}^T \mathbf{h} + b) = \frac{1}{1 + \exp(-\mathbf{w}^T \mathbf{h} - b)}$$

- ✓ Sigmoid输出单元常用于输出Bernoulli分布
- ✓ 适合二分类问题
- ✓ Sigmoid输出单元可采用交叉熵损失函数：

$$L(\mathbf{y}, \hat{\mathbf{y}}) = -\frac{1}{N} \sum_{n=1}^N (y^{(n)} \log \hat{y}^{(n)} - (1 - y^{(n)}) \log(1 - \hat{y}^{(n)}))$$

其中 $y^{(n)}$ 为真实值， $\hat{y}^{(n)}$ 为预测值， N 为样本数。



输出单元

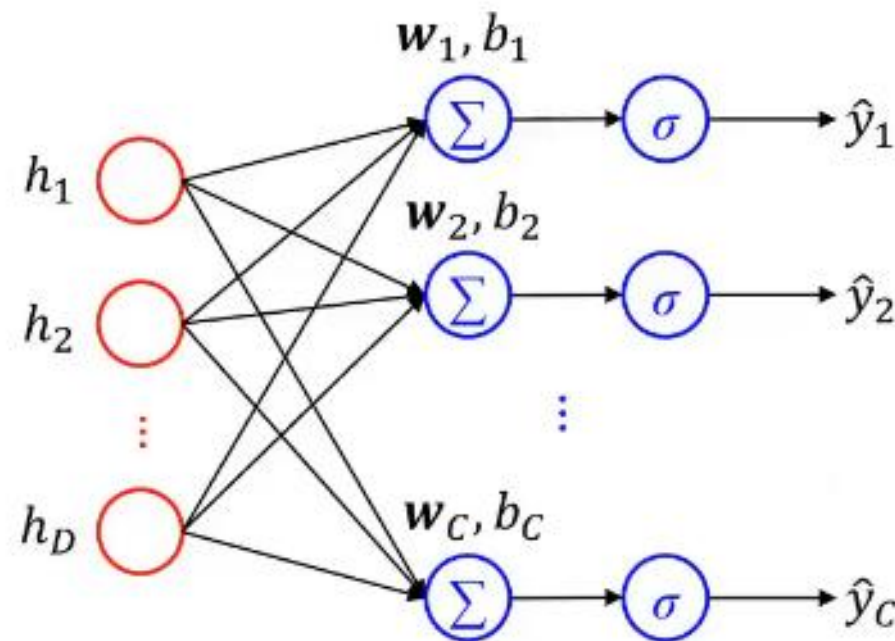
■ Softmax单元

$$\hat{y}_c = \text{softmax}(\mathbf{w}^T \mathbf{h} + \mathbf{b}) = \frac{\exp(\mathbf{w}_c^T \mathbf{h} + \mathbf{b}_c)}{\sum_{j=1}^C \exp(\mathbf{w}_j^T \mathbf{h} + \mathbf{b}_j)}$$

- ✓ Softmax输出单元常用于输出Multinoulli分布
- ✓ 适合多分类问题。
- ✓ Softmax输出单元可采用交叉熵损失函数：

$$L(\mathbf{y}, \hat{\mathbf{y}}) = -\frac{1}{N} \sum_{n=1}^N (\mathbf{y}^{(n)})^T \log \hat{\mathbf{y}}^{(n)}$$

其中 $\mathbf{y}^{(n)} = [y_1^{(n)}, y_2^{(n)}, \dots, y_C^{(n)}]^T$ 为真实标记向量值， $\hat{\mathbf{y}}^{(n)} = [\hat{y}_1^{(n)}, \hat{y}_2^{(n)}, \dots, \hat{y}_C^{(n)}]^T$ 为预测标记概率向量， N 为样本数， C 为标记数。



前馈神经网络的参数学习

■ 学习准则

- ✓ 假设神经网络使用交叉熵损失，对于一个样本 (x, y) ，其损失函数为：

$$L(y, \hat{y}) = -\mathbf{y}^T \log \hat{\mathbf{y}}$$

其中 $\mathbf{y} \in \{0,1\}^C$ 为标签 y 对应的one-hot向量表示。

- ✓ 给定一个训练集 $D = \{(\mathbf{x}^{(n)}, \mathbf{y}^{(n)})\}_{n=1}^N$ ，每个样本的特征向量 $\mathbf{x}^{(n)}$ 经过前馈神经网络的输出为 $\hat{\mathbf{y}}^{(n)}$ ，模型在数据集 D 的结构化风险函数为：

$$R(W, \mathbf{b}) = -\frac{1}{N} \sum_{n=1}^N L(\mathbf{y}^{(n)}, \hat{\mathbf{y}}^{(n)}) + \frac{1}{2} \lambda \|\mathbf{W}\|_F^2$$

其中 W 和 b 表示网络的全部参数， λ 为超参数，正则化项为矩阵Frobenius范数的平方

$$\|\mathbf{W}\|_F^2 = \sum_{l=1}^L \sum_{i=1}^{M_l} \sum_{j=1}^{M_{l-1}} (w_{ij}^{(l)})^2$$

前馈神经网络的参数学习

■ 梯度下降

- ✓ 基于学习准则和训练样本，网络参数可以通过梯度下降法进行学习，在每次迭代中第 l 层的参数 $\mathbf{W}^{(l)}$ 和 $\mathbf{b}^{(l)}$ 更新方式为：

$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \alpha \frac{\partial R(\mathbf{W}, \mathbf{b})}{\partial \mathbf{W}^{(l)}}$$

$$\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \alpha \frac{\partial R(\mathbf{W}, \mathbf{b})}{\partial \mathbf{b}^{(l)}}$$

其中 α 为学习率 (learning rate) 。

- ✓ 通过链式法则可以逐一对每个参数求偏导，但是效率低下
- ✓ 在神经网络的训练中经常使用反向传播算法来高效计算梯度

反向传播算法

反向传播算法

给定一个样本 (x, y) ，假设神经网络的输出为 $\hat{y}$ ，损失函数为 $L(y, \hat{y})$ ，采用梯度下降法计算损失函数关于每个参数的偏导数。

■ 如何计算前馈神经网络中参数的偏导数——反向传播 (Back Propagation, BP) 算法

考虑求第 l 层中参数 $W^{(l)}$ 和 $b^{(l)}$ 的偏导数，由于 $z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$ ，根据链式法则：

$$\frac{\partial L(y, \hat{y})}{\partial w_{ij}^{(l)}} = \frac{\partial z^{(l)}}{\partial w_{ij}^{(l)}} \frac{\partial L(y, \hat{y})}{\partial z^{(l)}}$$

$$\frac{\partial L(y, \hat{y})}{\partial b^{(l)}} = \frac{\partial z^{(l)}}{\partial b^{(l)}} \frac{\partial L(y, \hat{y})}{\partial z^{(l)}}$$

表达式一样，只需
计算一次

令 $\delta^{(l)} \triangleq \frac{\partial L(y, \hat{y})}{\partial z^{(l)}}$ 为第 l 层的误差项

反向传播算法

① 求 $\frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{w}_{ij}^{(l)}}$ ，由 $\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$ ：

$$\begin{aligned}\frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{w}_{ij}^{(l)}} &= \left[\frac{\partial z_1^{(l)}}{\partial w_{ij}^{(l)}}, \dots, \frac{\partial z_i^{(l)}}{\partial w_{ij}^{(l)}}, \dots, \frac{\partial z_{M_l}^{(l)}}{\partial w_{ij}^{(l)}} \right] \\ &= [0, \dots, a_i^{(l-1)}, \dots, 0] \in \mathbb{R}^{1 \times M_l}\end{aligned}$$

② 求 $\frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}}$ ，由 $\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$ ：

$$\frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}} = \mathbf{I}_{M_l} \in \mathbb{R}^{M_l \times M_l}$$

为 $M_l \times M_l$ 的单位矩阵。

反向传播算法

③ 求 $\delta^{(l)} \triangleq \frac{\partial L(y, \hat{y})}{\partial \mathbf{z}^{(l)}}$, 由 $\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$, $\mathbf{a}^{(l)} = f_l(\mathbf{z}^{(l)})$, 根据链式法则:

$$\frac{\partial L(y, \hat{y})}{\partial \mathbf{z}^{(l)}} = \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}} \frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{a}^{(l)}} \underbrace{\frac{\partial L(y, \hat{y})}{\partial \mathbf{z}^{(l+1)}}}_{= \delta^{(l+1)}}$$

其中

$$\frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}} = \frac{\partial f_l(\mathbf{z}^{(l)})}{\partial \mathbf{z}^{(l)}} = \text{diag}(f'_l(\mathbf{z}^{(l)})) \in \mathbb{R}^{M_l \times M_l} \quad \frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{a}^{(l)}} = (\mathbf{W}^{(l+1)})^T \in \mathbb{R}^{M_l \times M_{l+1}}$$

所以

$$\delta^{(l)} \triangleq \frac{\partial L(y, \hat{y})}{\partial \mathbf{z}^{(l)}} = f'_l(\mathbf{z}^{(l)}) \odot (\mathbf{W}^{(l+1)})^T \delta^{(l+1)} \in \mathbb{R}^{M_l}$$

第 l 层的误差项是第 $l+1$ 层的误差项的加权和, 然后再乘以该层的激活函数的梯度, 这就是误差的反向传播。

其中 $\odot$ 是点积, 表示每个元素相乘。

反向传播算法

计算出上面的三个偏导数之后。可得到第 l 层的梯度：

$$\frac{\partial L(y, \hat{y})}{\partial w_{ij}^{(l)}} = \frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}} \frac{\partial L(y, \hat{y})}{\partial \mathbf{z}^{(l)}} = \boldsymbol{\delta}_i^{(l)} \mathbf{a}_j^{(l-1)}$$

相当于向量 $\boldsymbol{\delta}^{(l)}$ 和向量 $\mathbf{a}^{(l-1)}$ 的外积的第 i, j 个元素，即：

$$\left[\frac{\partial L(y, \hat{y})}{\partial \mathbf{W}^{(l)}} \right]_{ij} = \left[\boldsymbol{\delta}^{(l)} \left(\mathbf{a}^{(l-1)} \right)^T \right]_{ij}$$

因此， $L(y, \hat{y})$ 关于第 l 层权重 $\mathbf{W}^{(l)}$ 的梯度为：

$$\frac{\partial L(y, \hat{y})}{\partial \mathbf{W}^{(l)}} = \boldsymbol{\delta}^{(l)} \left(\mathbf{a}^{(l-1)} \right)^T \in \mathbb{R}^{M_l \times M_{l-1}}$$

同理可得 $L(y, \hat{y})$ 关于第 l 层偏置 $\mathbf{b}^{(l)}$ 的梯度为：

$$\frac{\partial L(y, \hat{y})}{\partial \mathbf{b}^{(l)}} = \boldsymbol{\delta}^{(l)} \in \mathbb{R}^{M_l}$$

反向传播算法

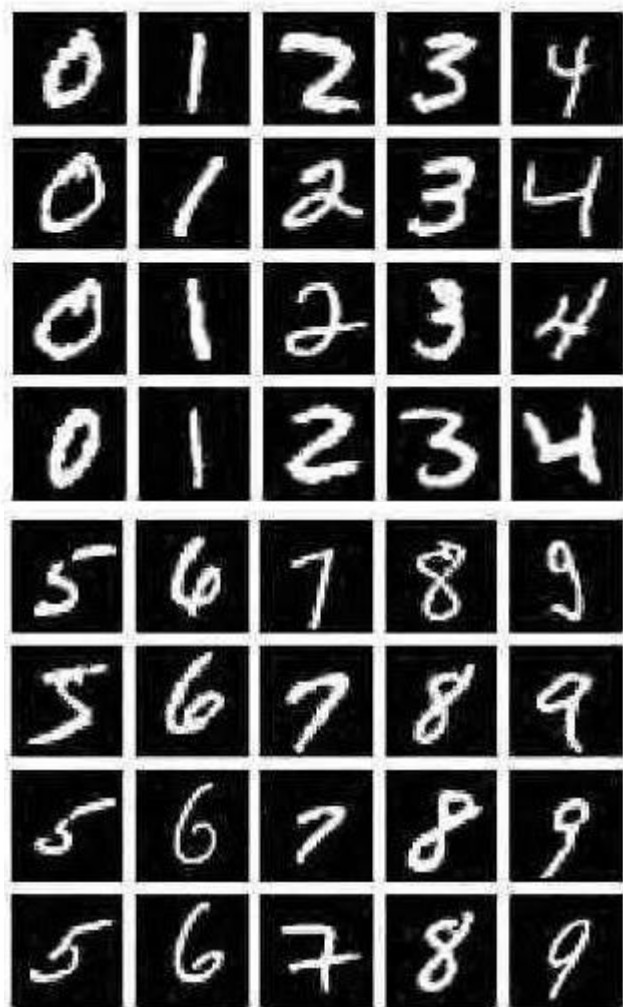
使用反向传播算法的前馈神经网络随机梯度下降训练过程

输入： 训练集 $D = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$, 验证集 V , 学习率 α , 正则化系数 λ , 网络层数 L , 神经元数量 $\{M_l\}_{l=1}^L$.

输出： W, b

```
1  随机初始化  $W, b$ ;  
2  repeat  
3      对训练集  $D$  中的样本随机重排序;  
4      For  $n = 1 \dots N$  do  
5          从训练集  $D$  中选取样本  $(\mathbf{x}^{(n)}, y^{(n)})$ ;  
6          前馈计算每一层的净输入  $\mathbf{z}^{(l)}$  和激活值  $\mathbf{a}^{(l)}$ , 直到最后一层;  
7          反向传播计算每一层的误差  $\delta^{(l)} = f'_l(\mathbf{z}^{(l)}) \odot ((\mathbf{W}^{(l+1)})^T \delta^{(l+1)})$ ; //最后一层的误差为  $\delta^{(L)} = f'_L(\mathbf{z}^{(L)}) \odot \frac{\partial L(\mathbf{y}, \hat{\mathbf{y}})}{\partial \hat{\mathbf{y}}}$   
          //计算每一层的梯度  
8           $\frac{\partial L(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} (\mathbf{a}^{(l-1)})^T$ ;  
9           $\frac{\partial L(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}$ ;  
          //更新参数  
10          $\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \alpha (\delta^{(l)} (\mathbf{a}^{(l-1)})^T + \lambda \mathbf{W}^{(l)})$ ;  
11          $\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \alpha \delta^{(l)}$ ;  
12     end;  
13 until 模型在验证集  $V$  上的错误率不再下降;
```

作业三：基于ANN的MNIST图像分类



- 推荐编程环境: Anaconda+Jupyter notebook+PyTorch
- 搭建不同层数的人工神经网络并查看模型性能变化
- 可以使用Scikit-learn、PyTorch、TensorFlow等不同的库进行实现对比。

Sources

- <https://www.youtube.com/watch?v=bfmFfD2RIcg>
Neural Network In 5 Minutes | What Is A Neural Network? 5mins
- <https://www.youtube.com/watch?v=CqOfi41LfDw>
Neural Networks Part. 1: Inside the Black Box 19mins
- <https://www.youtube.com/watch?v=oJNHXP0XDk>
Neural Network Architectures & Deep Learning 9mins
- <https://www.youtube.com/watch?v=3JQ3hYko51Y>
Neural Network 3D Simulation 3mins
- <https://www.youtube.com/watch?v=f0t-OCG79-U>
Convolutional Neural Network Visualization 2mins